

Software Architecture and Design Specification

Project: Blood Bank Management System

Version: 1.0

Authors: Team 9

Date: 3-10-2026

Status: Draft

Submission Link:<https://forms.gle/hrG1rscDZfJbD7WV7>

Revision History

Version	Date	Change	Summary
1.0	3-10-2026	Team 9	Initial draft of the Software Architecture and Design Specification for the Blood Bank Management System (BBMS).

Approvals

Role	Name	Signature/Date
Project Guide	Dr. Sudeepa Roy Dey Ma'am	
Team Representative	Shreelakshmi G Bhat	

1. Introduction

1.1 Purpose

This document specifies the software architecture and detailed design of the Blood Bank Management System (BBMS). It describes how the system is structured, how its components interact, and the design decisions behind them.

The BBMS is a new, self-contained web application that digitizes blood-bank operations, replacing manual, register-based record keeping. It supports donor registration and eligibility tracking, blood inventory management, blood requests and fulfilment between hospitals and blood banks, donation camps, SMS/email notifications, reporting, search and matching, and administrative management.

The architecture is designed to provide modularity, security, reliability, maintainability, and scalability while supporting the functional and non-functional requirements defined in the Software Requirements Specification (SRS) [1]. The system consists of a backend (Python/Django and/or JavaScript/Node.js), a relational database (PostgreSQL/MySQL), and a responsive web frontend. The frontend communicates with the backend through REST APIs using JSON over HTTPS.

This document records the chosen architecture pattern, technology stack, security approach, and the detailed designs (UML diagrams and API definitions) needed to implement, test, and maintain the system. It also serves as the design basis for the Software Test Plan (STP).

1.2 Scope

The architecture covers the following major system capabilities, which correspond to the system features specified in the SRS :

- User registration and authentication
- Role-based access control
- Donor registration and eligibility management
- Donation history
- Blood inventory management
- Blood-unit expiry and traceability
- Blood availability search
- Blood request and fulfilment
- Blood compatibility verification
- Donation camp management
- SMS/email notifications
- Reporting and analytics
- Donor/blood-bank search and matching
- Administrative management
- Audit logging
- Donation certificate generation

1.3 Audience

Audience	How they use this document
Development team	Understand component responsibilities, APIs, and the technology stack to implement the system
Software engineering instructor / evaluator	Assess the completeness and quality of the architecture and design deliverables
QA / testing team	Derive test cases from the design, sequence flows, and error-handling rules, and align them with the test plan
Future developers	Learn the structure and design rationale for extending the system
System administrators	Understand deployment, monitoring, and access control for operating the system
Project maintainers	Use the architecture as the reference for maintenance and change-impact analysis

1.4 Definitions

Term	Definition
BBMS	Blood Bank Management System
SRS	Software Requirements Specification
STP	Software Test Plan
RTM	Requirements Traceability Matrix
FR	Functional Requirement
NFR	Non-Functional Requirement
RBAC	Role-Based Access Control, restricting features and data according to a user's role
OTP	One-Time Password, a temporary code used to verify a user's identity
JWT	JSON Web Token, a compact token used to authenticate API requests
API	Application Programming Interface

REST	Representational State Transfer, an architectural style for web APIs
JSON	JavaScript Object Notation, the data format used by the APIs
HTTPS	HTTP over encrypted transport, used for all client-server communication
DB	Database
UML	Unified Modeling Language, a standard notation for software diagrams
ADR	Architecture Decision Record, a short record of an architectural decision and its rationale
ABO/Rh	The blood-group system used to determine transfusion compatibility
Blood Unit	A uniquely identified quantity of collected and stored blood, tracked through its lifecycle
Compatibility	Rules determining which donor blood groups can safely be given to which recipient blood groups
Eligibility Interval	The configured minimum time a donor must wait between successive donations
Donation Camp	An organized event where blood is collected from donors outside the blood bank

2. Document Overview

2.1 How to use this document

This Software Architecture and Design Specification (SASD) describes the architecture and design of the Blood Bank Management System (BBMS). It provides the structural and

technical details required to understand the system architecture, major components, interactions, interfaces, security considerations, and design decisions. The Architecture section describes the system structure, components, architecture pattern, technology stack, and security architecture. The Design section describes system interactions, sequence diagrams, API design, error handling, logging, monitoring, and user experience considerations. This document should be used along with the Software Requirements Specification (SRS), Software Test Plan (STP), and Requirements Traceability Matrix (RTM).

2.2 Related Documents

Related Document	Purpose
Software Requirements Specification (SRS)	Defines the functional and non-functional requirements of the Blood Bank Management System (BBMS).
Software Test Plan (STP)	Defines the testing strategy, test activities, test environment, risks, deliverables, and test criteria for BBMS.
Requirements Traceability Matrix (RTM)	Maps system requirements to their corresponding design and test elements.
Project Proposal / Synopsis	Provides the project overview, objectives, scope, team responsibilities, and development plan.

3. Architecture

3.1 Goals & Constraints

Goals:

Secure and reliable blood-bank operations, modularity, maintainability, scalability, accurate blood inventory tracking, prevention of incorrect/expired blood allocation, and blood availability searches within approximately 2–3 seconds.

Constraints:

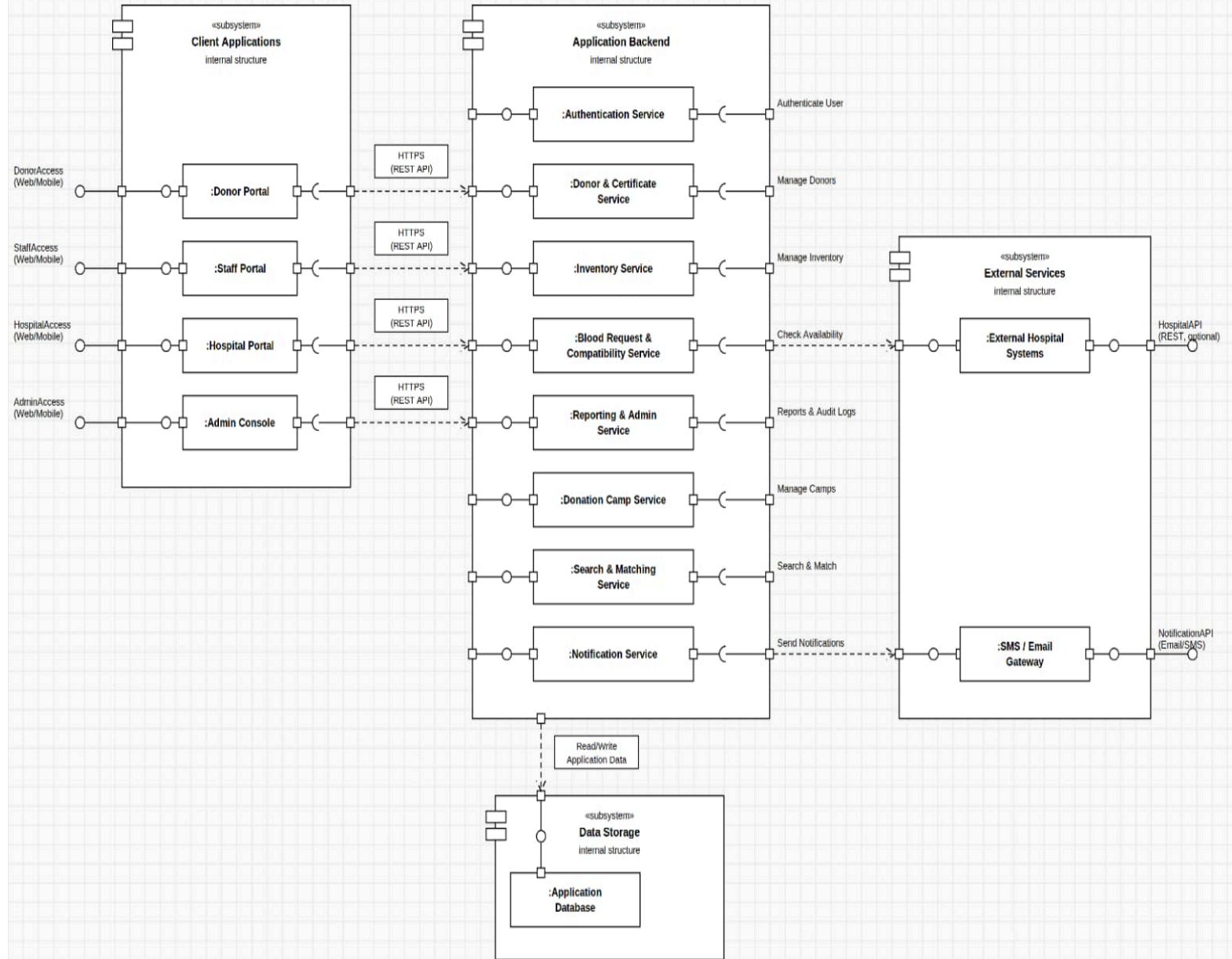
Web-based application, relational database, role-based access control, HTTPS-secured communication, configurable operational rules, semester-based development timeline, and optional integration with SMS/email gateways and external hospital systems.

3.2 Stakeholders & Concerns

1. **Donors:** Simple registration, eligibility information, and donation history.
2. **Hospital/Patients:** Fast blood search, request tracking, and emergency availability.
3. **Blood Bank Staff:** Accurate inventory, stock levels, expiry tracking, and request processing.
4. **Administrators:** User management, reports, audit logs, and system configuration.
5. **Development Team:** Modularity, maintainability, clear APIs, and easy testing.
6. **Instructor/Evaluator:** Correct implementation of requirements and traceability.

3.3 Component (UML) Diagram

Blood Bank Management System
UML Component Diagram



3.4 Component Descriptions

- 1. Web Frontend:** Provides role-specific interfaces for donors, hospital/patients, blood-bank staff, and administrators.
- 2. REST API Layer:** Receives requests from the frontend and returns responses through REST/JSON APIs.
- 3. Authentication & RBAC:** Handles authentication, OTP verification, authorization, and role-based access.
- 4. User Management:** Handles user registration, login, profiles, and account management.
- 5. Donor Management:** Manages donor details, eligibility, and donation history.
- 6. Inventory Management:** Manages blood-unit records, stock levels, expiry tracking, disposal, filtering, and traceability. It also supports availability checking, low-stock monitoring, and validation to prevent expired or already-reserved blood units from being allocated.
- 7. Blood Request & Availability:** Handles blood searches, requests, reservation, approval, compatibility verification, and fulfilment.
- 8. Donation Camp Management:** Handles camp creation, publishing, and donor registration.
- 9. Notification Service:** Sends SMS/email alerts, low-stock notifications, and emergency notifications.
- 10. Reporting & Analytics:** Generates reports, statistics, and dashboards.
- 11. Search & Matching:** Supports donor, blood-bank, and blood-availability matching.
- 12. Admin Module:** Manages users, blood banks, staff, roles, and system configuration.
- 13. Certificate Service:** Generates donation certificates.
- 14. Audit Logger:** Records important operations such as inventory changes, blood requests, approvals, allocations, and administrative actions with timestamps and responsible users.
- 15. PostgreSQL Database:** Stores persistent BBMS data including users, donors, blood banks, blood units, donations, blood requests, allocations, notifications, audit logs, and certificates.

3.5 Chosen Architecture Pattern and Rationale

-> Layered Modular Monolithic Architecture is chosen for BBMS.

->The architecture separates the system into the Presentation Layer, API/Controller Layer, Business Logic Layer, Data Access Layer, and PostgreSQL Database.

->This architecture is suitable because BBMS contains multiple logically separate modules such as User Management, Donor Management, Inventory Management, Blood Requests, Notifications, and Reporting.

It provides:

- Clear separation of concerns
- Easier testing
- Easier maintenance
- Controlled database access
- Clear API boundaries
- Ability to extend individual modules

Microservices are not selected because they would introduce additional deployment, networking, monitoring, and service-management complexity that is unnecessary for the current four-member academic project.

3.6 Technology Stack & Data Stores

Layer	Technology
Frontend	HTML, CSS, JavaScript / React
Backend	Python + Django REST Framework
API	REST + JSON
Database	PostgreSQL
Authentication	JWT / Secure Session Authentication
Notifications	SMS/Email Gateway
Version Control	Git + GitHub

Deployment	Docker-compatible Deployment
------------	------------------------------

Data Store: PostgreSQL is used as the relational database for storing users, donors, blood banks, blood units, inventory, donations, blood requests, allocations, notifications, audit logs, and certificates.

3.7 Risks & Mitigations

Risk	Mitigation
Incorrect or expired blood unit being allocated	Perform compatibility, expiry, and reservation-status validation before fulfilment.
Inventory inconsistency	Update inventory after allocation/fulfilment and maintain audit logs.
Unauthorized inventory modification	Use RBAC and authenticated API access.
Blood stock becoming critically low	Provide configurable low-stock alerts.
Database/network failure	Use transactional database operations and proper error handling.
Incorrect blood-unit traceability	Assign unique identifiers and record inventory changes and allocations.

3.8 Traceability to Requirements

Requirement	Architecture Component
FR8 – Add, update, and remove blood-unit records	Inventory Management
FR9 – Track blood-unit expiry and prevent expired units from being issued	Inventory Management

FR10 – Provide real-time stock levels by blood group and blood-bank location	Inventory Management
FR11 – Generate low-stock alerts for configured blood groups	Inventory Management + Notification Service
FR30 – Maintain blood-unit traceability using unique identification	Inventory Management + Audit Logger
FR33 – Record disposal of expired, damaged, or unusable blood units	Inventory Management + Audit Logger
FR34 – Filter and sort inventory by blood group, expiry date, collection date, and location	Inventory Management

3.9 Security Architecture

BBMS uses authentication, role-based access control, HTTPS, authorization, and audit logging to protect user, donor, patient, and blood-bank data.

For the Inventory Management module:

- **Unauthorized access → RBAC:** Only authorized Blood Bank Staff and Administrators can perform inventory operations.
- **Credential theft → Secure password storage + OTP verification:** User credentials are securely stored, and OTP verification provides an additional layer of identity verification.
- **Unauthorized API calls → Authentication tokens/sessions:** API requests require valid authentication before accessing protected resources.
- **Data interception → HTTPS:** Communication between the client and server is secured using HTTPS.
- **Data leakage → Access restrictions + encryption:** Access to sensitive information is restricted according to user roles and protected through appropriate security mechanisms.
- **Malicious modification → Authorization + Audit Logs:** Important inventory changes are authorized and recorded with the responsible user and timestamp.
- **Incorrect blood allocation → Compatibility validation:** Blood-group compatibility is checked before a blood unit is allocated.
- **Expired blood allocation → Expiry validation:** Expired blood units are identified and prevented from being issued.

4. Design

4.1 Design Overview

The system is designed as a modular web application.

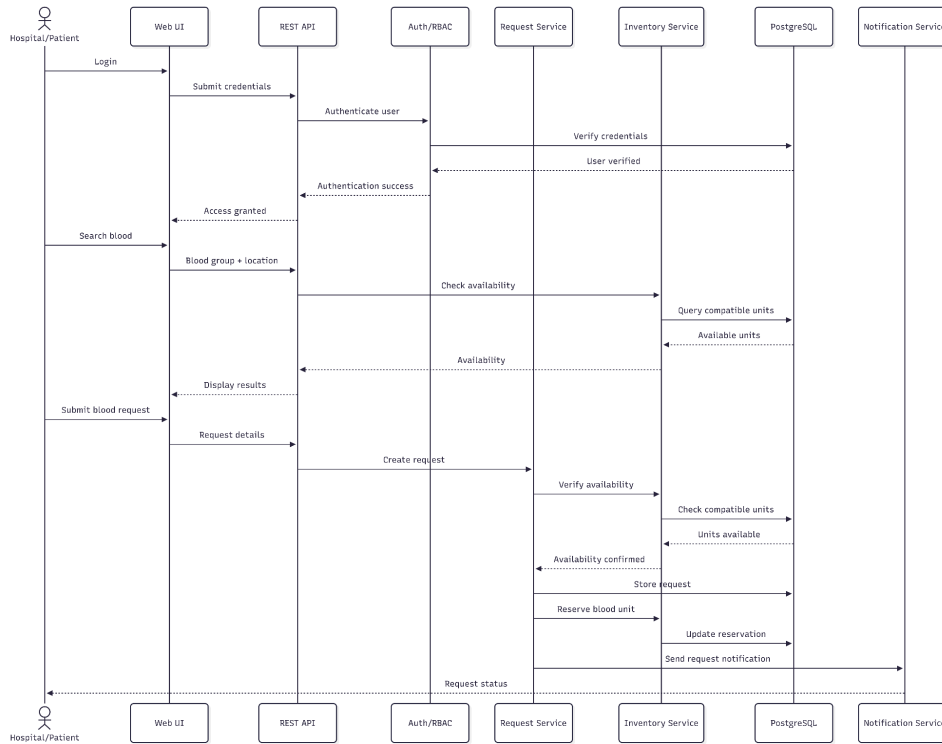
Each module owns a specific business responsibility while communicating through well-defined service/API interfaces.

The major modules are:

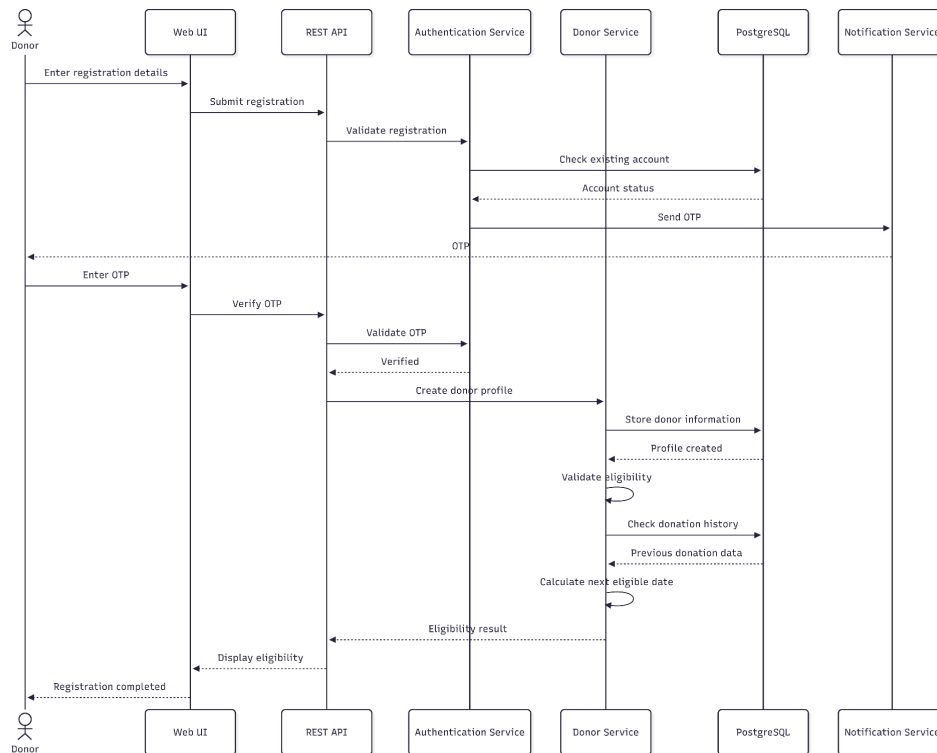
1. User Management
2. Donor Management
3. Inventory Management
4. Blood Request & Availability
5. Donation Camp Management
6. Notifications
7. Reporting & Analytics
8. Search & Matching
9. Administration
10. Certificate Generation
11. Audit Logging

4.2 UML Sequence Diagrams

1. Blood Request & Fulfilment



2. Donor Registration & Eligibility



4.3 API Design

API 1 - Blood Availability

Endpoint

GET /api/blood/availability

Parameters

blood_group

location

Example

GET /api/blood/availability?blood_group=O%2B&location=Bangalore

Response

```
{
  "blood_group": "O+",
  "location": "Bangalore",
  "available_units": 12,
  "status": "available"
}
```

API 2 - Create Blood Request

POST /api/blood-requests

Request

```
{
  "blood_group": "O+",
  "quantity": 2,
  "location": "Bangalore",
  "priority": "emergency"
}
```

Response

```
{  
  "request_id": "REQ1024",  
  "status": "pending",  
  "message": "Blood request submitted successfully"  
}
```

Possible errors

400 Invalid request

401 Unauthorized

403 Insufficient permissions

404 Blood group/location unavailable

409 Request conflict

4.4 Error Handling, Logging & Monitoring

The system shall use standardized HTTP status codes:

Code	Meaning
200	Successful request
201	Resource created
400	Invalid request
401	Authentication required
403	Unauthorized action
404	Resource not found

409	Conflict
500	Internal server error

Logging

The system shall log:

- Login/security events
- Blood donations
- Inventory changes
- Blood requests
- Approvals/rejections
- Blood allocation
- Administrative actions

Sensitive information such as passwords and authentication tokens shall not be written to logs.

The SRS explicitly requires audit logs for donations, requests, inventory changes, approvals and other critical actions.

Monitoring

Monitor:

- API response time
- Blood search response time
- Failed requests
- Authentication failures
- Inventory inconsistencies
- Notification failures
- System availability

4.5 UX Design

The UI shall provide role-specific dashboards:

Donor Dashboard

- Profile
- Eligibility

- Next eligible donation date
- Donation history
- Camp registration
- Notifications
- Certificates

Hospital/Patient Dashboard

- Blood search
- Submit request
- Emergency request
- Request status
- Request history

Staff Dashboard

- Blood inventory
- Donors
- Requests
- Donation camps
- Stock alerts

Admin Dashboard

- Users
- Blood banks
- Staff
- Reports
- Analytics
- Audit logs
- Configuration

The SRS requires role-specific dashboards and a responsive interface supporting desktop and mobile devices.

4.6 Open Issues & Next Steps

Open Issues

- Final selection between Django and Node.js if not yet locked
- Final database schema
- Selection of SMS/email gateway
- UI design/wireframes
- Exact deployment environment

- External hospital API integration

Next Steps

1. Finalize technology stack
2. Finalize database ER diagram
3. Implement authentication/RBAC
4. Implement donor module
5. Implement inventory module
6. Implement request workflow
7. Implement camps and notifications
8. Implement reports
9. Integrate modules
10. Perform system and integration testing

5. Appendices

5.1 Glossary:

Term	Meaning
BBMS	Blood Bank Management System
FR	Functional Requirement
NFR	Non-Functional Requirement
RBAC	Role-Based Access Control
OTP	One-Time Password
Blood Unit	Uniquely identified collected blood quantity
Eligibility Interval	Minimum period between donations

Compatibility	Rules for safe blood allocation
---------------	---------------------------------

5.2 References:

- Team 9 Blood Bank Management System SRS, Version 1.0
- Team 9 Blood Bank Management System Project Proposal
- IEEE Software Requirements Specification template provided for the course
- Course Software Engineering material

5.3 Tools:

- StarUML
- draw.io
- Swagger
- mermaid